



OS MOTORES: HARNESS, SKILLS, MCPS E WORKTREES

Uma leitura prática sobre o desenvolvimento orientado a agentes

Heverton Eduardo Peres

ENGENHEIRO DE SOFTWARE & MAKER

Heverton Eduardo Peres

Os Motores: Harness, Skills, MCPs e Worktrees

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

São Paulo
2026

Dados Internacionais de Catalogação na Publicação (CIP)

P604m PERES, Heverton Eduardo

Os Motores: Harness, Skills, MCPs e Worktrees / Heverton Eduardo
Peres. – São Paulo : Fábrica Agêntica de Livros,
2026.

31 p. ; 21 cm.

ISBN 978-65-00000-40-5

1. Motores. 2. Harness. 3. Skills. 4. MCPs. I. Título.

CDD 001.2

Sumário

Os Motores: Harness, Skills, MCPs e Worktrees	6
Capítulo 5: Os Motores: Harness, Skills, MCPs e Worktrees	7
Introdução	7
Explica	7
Ilustra	8
Técnica	9
O Loop do Harness em Código	9
Configuração de Ferramentas com MCP	10
Skills como Procedimentos Empacotados	11
Worktrees em Ação	11
O Manifesto de Skills do Time	12
O Modelo de Sequenciamento de Skills	13
O Modelo de Avaliação de Ferramentas por Critério Ponderado	13
O Estudo de Caso: Montando o Laboratório de Motores	14
O Benchmark de Motores como Rotina Contínua	15
O Protocolo de Registro de MCPs	15
O Protocolo de Registro de MCPs	16
O Modelo de Decisão de Aquisição de Ferramentas	16
O Playbook de Diagnóstico de Motores	17
O Modelo de Inventário de Skills com Custos	17
O Registro de Ativação de Skills	18
O Test-First Como Régua do Harness	19
O Ciclo de Vida de uma Skill	20
O Modelo de Custo Total do Laboratório	20
A Governança do Ecossistema	21
O Radar do Ecossistema	21
O Ecossistema em Números	21
O Protocolo de Entrada de Nova Ferramenta	22
Passos para Montar o Laboratório de Motores	22
Aplica	22
Conclusão	23
Por que este capítulo importa	23
Conceitos-chave deste capítulo	23
Checklist para aplicar hoje	24

Perguntas que você deve se fazer	24
Glossário rápido	24
O erro mais comum nesta fase	25
Um exemplo concreto para fixar	25
A rotina de quem já opera assim	25
O que não fazer: os anti-padrões mais comuns	26
Como medir o progresso na prática	26
O papel do líder neste capítulo	26
Perguntas frequentes honestas	27
Um convite para a prática deliberada	27
Síntese para levar com você	28
De onde veio a necessidade desta mudança	28
Conversando com quem resiste	28
O dia a dia no detalhe	29
O custo invisível que decide tudo	29
Uma visão de longo prazo	29
Próximos Passos	30

Os Motores: Harness, Skills, MCPs e Worktrees

Uma leitura direta e prática para quem quer levar o desenvolvimento orientado a agentes a sério — sem jargão acadêmico, com exemplos aplicáveis.

Capítulo 5: Os Motores: Harness, Skills, MCPs e Worktrees

Introdução

No Capítulo 4, você desenhou a cartografia do domínio: vocabulário ubíquo, interfaces primeiro e ADRs — o mapa que orienta os agentes. Agora chegou a hora de conhecer os motores que colocam o mapa em movimento: o harness agêntico que orquestra, as skills que especializam, os MCPs que conectam ferramentas e as worktrees que isolam territórios de trabalho.

Este capítulo é o mais operacional da Parte III. Você vai aprender a arquitetura harness → LLM → ferramentas, entender como skills empacotam procedimentos reutilizáveis, como MCPs exponibilizam ferramentas via protocolo padrão, e como worktrees permitem despacho paralelo seguro de agentes. Você vai sair com um laboratório de motores configurável — e o hábito de test-first como régua do build.

Explica

Um harness agêntico é o ambiente de execução que dá ao LLM a capacidade de **agir**, não apenas de responder. A distinção é estrutural: um chat responde; um harness executa um loop — o modelo propõe uma ação, a ferramenta executa, o resultado volta para o modelo, que propõe a próxima ação. É esse loop de execução com feedback que transforma um LLM em um agente.

O framework conceitual é sempre o mesmo: harness → LLM → ferramentas. O harness é o processo que gerencia o loop (bash, edição de arquivos, testes, navegação). O LLM é o cérebro que decide as ações dentro do loop. E as ferramentas são os músculos — comandos, scripts, APIs — que o LLM aciona. A pesquisa da Anthropic demonstrou que a qualidade do scaffolding do harness (as ferramentas e sua interface) tem impacto tão grande quanto o modelo na resolução de problemas reais.

As skills são o próximo nível: procedimentos empacotados que especializam o agente. Uma skill é um conjunto de instruções reutilizáveis — um fluxo de trabalho, regras de domínio, templates — que o agente carrega quando o contexto exige. No SDLC AI-first, as skills são os

operários especializados: a skill de redação, a skill de revisão, a skill de testes. Elas codificam o conhecimento que uma equipe humana levaria anos para acumular.

Os MCPs (Model Context Protocol) resolvem um problema de conectividade: como o agente acessa os sistemas de dados e ferramentas da organização. Antes do MCP, cada integração era um adaptador customizado. Com o MCP, um protocolo padrão conecta o harness a qualquer fonte — repositório, banco de dados, sistema de tickets — com autenticação e contexto seguros. Para o ciclo de vida, o MCP é o sistema de radar da torre: conecta o agente aos dados de que ele precisa sem arrastar o mundo inteiro para o contexto.

As worktrees de git são o instrumento de isolamento. Cada agente — ou cada lote de agentes — trabalha em uma cópia isolada do repositório (uma worktree), e os resultados são integrados por merge controlado. Isso elimina a classe inteira de conflitos de edição concorrente que derrubam equipes agênticas. Worktree não é conveniência; é a célula de contenção do build paralelo.

O test-first completa o quadro operacional. A régua do build é: escrever o teste vermelho antes do código que o faz passar. No contexto agêntico, o teste é o contrato de conclusão: o agente termina quando a suíte passa, não quando ele acha que está pronto. A verificação deixa de ser opinião e vira execução — um diffs cujo critério de pronto é mensurável.

A combinação dos quatro instrumentos é o que separa o laboratório do caos: harness gerencia o loop, skills especializam o comportamento, MCPs conectam os dados e worktrees isolam a execução. Cada um resolve uma classe específica de falha, e juntos definem o ambiente onde o SDLC AI-first opera de verdade.

Ilustra

A torre de controle moderna não é só um prédio com radar — é uma arquitetura de sistemas. O radar (MCP) conecta a torre aos dados de voo. Os procedimentos operacionais padrão (skills) dizem ao controlador exatamente o que fazer em cada situação. As cabines de controle (worktrees) isolam cada controlador em seu setor, com suas telas e seus dados — ninguém edita a tela do vizinho. E a torre em si (harness) orquestra tudo, rodando o loop de observar-decidir-agir continuamente.

[Diagrama do capítulo omitido neste formato.]

Como Comandante de Operações de Software, você percebe o padrão: cada motor responde a uma pergunta operacional. O harness responde “como o agente age?”. As skills respondem “o que o agente sabe fazer?”. Os MCPs respondem “a que o agente tem acesso?”. As worktrees respondem “onde o agente pode pisar?”.

Técnica

O Loop do Harness em Código

O coração do harness é o loop agêntico. A implementação abaixo — deliberadamente minimalista — mostra a anatomia: o modelo propõe, a ferramenta executa, o resultado volta.

```
from dataclasses import dataclass, field
from typing import Callable, Dict, List

@dataclass
class Tool:
    nome: str
    funcao: Callable[[str], str]
    descricao: str = ""

class HarnessSimples:
    def __init__(self, ferramentas: List[Tool]) -> None:
        self.ferramentas: Dict[str, Tool] = {t.nome: t for t in
ferramentas}
        self.historico: List[str] = field(default_factory=list)

    def agir(self, acao: str, argumento: str) -> str:
        if acao not in self.ferramentas:
            return f"ERRO: ferramenta '{acao}' inexistente"
        self.historico.append(f"{acao}({argumento})")
        return self.ferramentas[acao].funcao(argumento)

    def loop(self, modelo_acao) -> List[str]:
        """Simula o loop: modelo -> ferramenta -> feedback -> proxima
acao."""
        resultado = ""
        passos = 0
        while passos < 10:
            acao, argumento = modelo_acao(resultado)
            if acao == "FIM":
                break
            resultado = self.agir(acao, argumento)
            passos += 1
        return self.historico
```

```
def ler_arquivo(caminho: str) -> str:
    try:
        with open(caminho, encoding="utf-8") as f:
            return f.read()[:200]
    except OSError as exc:
        return f"ERRO: {exc}"

def escrever_arquivo(conteudo: str) -> str:
    return f"OK: {len(conteudo)} caracteres em buffer (nao gravado neste exemplo)"

harness = HarnessSimples([
    Tool("ler", ler_arquivo, "le um arquivo"),
    Tool("escrever", escrever_arquivo, "escreve conteudo"),
])

# Simulacao de um "modelo" burro mas funcional
def modelo_exemplo(ultimo_resultado: str):
    if "erro" in ultimo_resultado.lower():
        return "FIM", ""
    return "ler", "README.md"

print(harness.loop(modelo_exemplo))
```

A moral do trecho: o harness não é mágica — é um loop disciplinado de ação e feedback. É essa estrutura que permite ao agente tentar, falhar e corrigir dentro de um ambiente controlado.

Configuração de Ferramentas com MCP

O MCP padroniza a conexão com dados. A configuração abaixo, no formato `.mcp.json`, registra servidores MCP de arquivos e banco de dados:

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/projeto/codigo",
        "/projeto/specs"
      ]
    }
  },
}
```

```

"sqlite": {
  "command": "npx",
  "args": ["-y", "mcp-server-sqlite", "/projeto/data/estado.db"]
}
}
}

```

A vantagem prática para o ciclo de vida: o agente consulta o banco de estado da esteira (fase atual, artefatos produzidos) via MCP, sem que o orquestrador carregue tudo no contexto. O protocolo faz a entrega sob demanda — a essência do lean context.

Skills como Procedimentos Empacotados

Uma skill é um arquivo de instruções que o agente carrega sob demanda. O esqueleto abaixo mostra a anatomia:

```

# Skill: revisor-espec

## Quando usar
Quando um artefato de spec for produzido na Fase 2 do ciclo.

## Procedimento
1. Leia a spec e extraia os requisitos R1..Rn.
2. Para cada requisito, verifique se existe criterio de aceite testavel.
3. Se algum requisito nao tiver criterio, reprove com a lista de falhas.
4. Se todos tiverem, aprove e registre o parecer com evidencia.

## Regras
- Nunca reescreva a spec; apenas aprove ou reprove com lista objetiva.
- Evidencia antes de afirmacao: cite a linha do criterio em cada parecer.

```

O valor da skill está na reutilização: o mesmo procedimento roda em todas as specs, com a mesma régua — eliminando a variação entre revisores.

Worktrees em Ação

O isolamento de território por worktree é trivial no git, mas muda o jogo agêntico:

```

# criar worktree isolada para o agente A (modulo de pagamentos)
git worktree add ../wt-pagamentos -b feature/pagamentos main

# criar worktree isolada para o agente B (modulo de inventario)
git worktree add ../wt-inventario -b feature/inventario main

```

```
# ao concluir, cada agente abre um PR; o merge é controlado na raiz
git worktree list
```

Cada worktree é um repositório completo com branch própria. Dois agentes nunca editam o mesmo arquivo físico. E o merge — autorizado pelo controlador humano — é o ponto único de integração.

O Manifesto de Skills do Time

Skills precisam de descoberta: o agente precisa saber qual skill carregar em cada fase. O manifesto abaixo declara o catálogo de skills — quando usar, qual fase serve e qual artefato produz.

```
{
  "skills": [
    {
      "nome": "revisor-espec",
      "fase": "spec",
      "disparo": "artefato spec_executavel produzido",
      "artefato_saida": "parecer_espec.md"
    },
    {
      "nome": "estrategista-pilares",
      "fase": "design",
      "disparo": "capitulo de design iniciado",
      "artefato_saida": "pilares.json"
    },
    {
      "nome": "redator-eita",
      "fase": "build",
      "disparo": "pilares aprovados",
      "artefato_saida": "capitulo.md"
    },
    {
      "nome": "verificador-adversarial",
      "fase": "verificar",
      "disparo": "diff do build",
      "artefato_saida": "parecer_verificacao.md"
    }
  ],
  "regra": "uma skill por fase; o orquestrador consulta o manifesto antes de delegar"
}
```

O manifesto é o catálogo de procedimentos da torre: quando o controlador precisa de um procedimento, consulta o catálogo — nunca improvisa.

O Modelo de Sequenciamento de Skills

As skills não agem isoladas — elas se encadeiam em um fluxo. O modelo de sequenciamento declara a ordem e as dependências entre skills de uma fase:

Skill	Fase	Depende de	Alimenta
Estrategista	Design	Cartografia	Redator
Redator	Build	Estrategista	Verificador
Verificador	Verificar	Redator	Revisor
Revisor	Verificar	Verificador	Compilador

O sequenciamento evita o erro clássico do despacho: a skill que roda antes de sua entrada existir. A esteira consulta o sequenciamento antes de cada ativação — a skill só carrega quando o que ela precisa está pronto.

O Modelo de Avaliação de Ferramentas por Critério Ponderado

O laboratório precisa comparar ferramentas de forma justa — e a comparação exige critérios ponderados, não impressão. O modelo abaixo pontua cada ferramenta em cinco dimensões e devolve o ranking:

```
CRITERIOS = {
    'qualidade_saida': 0.3,
    'custo_contexto': 0.25,
    'confiabilidade': 0.2,
    'facilidade_uso': 0.15,
    'comunidade': 0.1,
}

def rankear_ferramentas(ferramentas):
    resultado = []
    for nome, notas in ferramentas.items():
        score = sum(notas[c] * CRITERIOS[c] for c in CRITERIOS)
        resultado.append({'ferramenta': nome, 'score': round(score, 2)})
    return sorted(resultado, key=lambda x: x['score'], reverse=True)

ferramentas = {
    'harness-alpha': {'qualidade_saida': 4, 'custo_contexto': 3,
```

```
'confiabilidade': 4, 'facilidade_uso': 3, 'comunidade': 2},
    'harness-beta': {'qualidade_saida': 3, 'custo_contexto': 4,
'confiabilidade': 3, 'facilidade_uso': 4, 'comunidade': 4},
}
print(rankear_ferramentas(ferramentas))
```

O ranking por score substitui o debate de opinião pela tabela: se o harness-alpha ganha por pouco em qualidade de saída, mas consome muito contexto e tem comunidade pequena, o harness-beta pode ser a escolha de longo prazo. Os pesos são calibrados pela própria organização — uma equipe pequena valoriza facilidade de uso mais que comunidade. O ranking é um ponto de partida para conversa, não um veredito automático, mas obriga a conversa a ser sobre os números.

O Estudo de Caso: Montando o Laboratório de Motores

A escolha do harness é uma decisão de arquitetura — e o Comandante a faz com critérios explícitos. O modelo abaixo é a matriz de avaliação que compara opções antes de investir:

Critério	Peso	O que avalia
Loop de execução	30%	Ferramentas persistentes, feedback ao modelo, retry
Conectividade MCP	25%	Facilidade de registrar servidores de dados
Skills	20%	Suporte a procedimentos empacotados sob demanda
Worktrees	15%	Isolamento nativo de território de execução
Custo/contexto	10%	Eficiência de tokens por operação

A matriz vira score — e o score vira decisão:

```
def avaliar_harness(criterios: dict) -> dict:
    pesos = {"loop": 0.30, "mcp": 0.25, "skills": 0.20,
            "worktrees": 0.15, "custo": 0.10}
    total = sum(criterios.get(k, 0) * pesos[k] for k in pesos)
    return {"score": round(total, 2), "veredito": "adotar" if total >= 0.7
    else "avaliar mais"}

HARNESS_A = avaliar_harness({"loop": 0.9, "mcp": 0.8, "skills": 0.9,
                             "worktrees": 0.7, "custo": 0.6})
HARNESS_B = avaliar_harness({"loop": 0.6, "mcp": 0.5, "skills": 0.4,
```

```

                                "worktrees": 0.3, "custo": 0.8})
print(f"Harness A: {HARNESS_A}")
print(f"Harness B: {HARNESS_B}")

```

A avaliação por critérios evita o erro clássico de escolher harness por moda ou por preço — o loop de execução pesa três vezes mais que o custo, porque é o coração do agente.

O Benchmark de Motores como Rotina Contínua

O laboratório de motores não se monta uma vez — ele precisa de benchmark contínuo porque os motores mudam, os prompts mudam e o desempenho degrada. O script abaixo roda um conjunto fixo de tarefas contra cada motor e gera um placar comparativo:

```

import json

TAREFAS_BENCHMARK = [
    {'id': 'T1', 'tipo': 'refatoracao', 'criticidade': 'media'},
    {'id': 'T2', 'tipo': 'testes', 'criticidade': 'alta'},
    {'id': 'T3', 'tipo': 'documentacao', 'criticidade': 'baixa'},
    {'id': 'T4', 'tipo': 'debug', 'criticidade': 'alta'},
]

def rodar_benchmark(motor, resultados):
    placar = {t['id']: resultados.get(motor, {}).get(t['id']) for t in
TAREFAS_BENCHMARK}
    taxa = sum(1 for v in placar.values() if v == 'ok') / len(placar)
    return {'motor': motor, 'taxa_sucesso': taxa, 'por_tarefa': placar}

print(rodar_benchmark('motor-alpha', {'motor-alpha': {'T1': 'ok', 'T2':
'ok', 'T3': 'falha', 'T4': 'ok'}}))

```

O placar vira dado de governança: quando um motor novo entra no laboratório, ele é avaliado contra o mesmo benchmark — nada de comparar maçãs com laranjas. E quando a taxa de sucesso de um motor cai abaixo do limiar, o protocolo dispara: ou o prompt do contrato precisa de atualização, ou o motor saiu do padrão. O benchmark também protege contra a síndrome do novo brinquedo: a organização troca de motor por evidência, não por hype.

O Protocolo de Registro de MCPs

Vamos montar o laboratório de motores do zero, em sequência operacional. O cenário: uma equipe de 5 engenheiros que quer começar o SDLC AI-first sem quebrar o que já funciona.

1. **Semana 1 — Harness:** escolher o harness, configurar o loop de execução (bash, edição, testes) e o MCP de filesystem com escopo de leitura.

2. **Semana 2 — Test-first:** adotar o teste vermelho antes do código em um fluxo piloto — a régua do build.
3. **Semana 3 — Worktrees:** configurar worktree por agente e o merge controlado.
4. **Semana 4 — Skill piloto:** empacotar o procedimento de revisão de spec como skill e registrar no manifesto.
5. **Semana 5 — Radar:** adicionar o revisor adversarial independente nos PRs dos agentes.

A sequência é deliberada: cada semana constrói sobre a anterior, e nenhum passo exige derrubar o processo existente. O laboratório nasce paralelo à operação — e a operação adota os motores quando eles provam valor.

O Protocolo de Registro de MCPs

O acesso a dados via MCP precisa de protocolo — quem pode conectar a qual fonte, com qual escopo. O registro abaixo é o contrato de conectividade do ecossistema:

```
mcp_registro:
  filesystem:
    escopo_leitura: [specs, contratos, dossie]
    escopo_escrita: []
    agentes_permitidos: [analista, arquiteto]
  banco_estado:
    escopo_leitura: [fase, artefatos, metricas]
    escopo_escrita: [transicao_fase]
    agentes_permitidos: [orquestrador]
  repositorio:
    escopo_leitura: [codigo, testes]
    escopo_escrita: [worktree_do_agente]
    agentes_permitidos: [agente-build, agente-revisor]
```

O registro de MCPs é o mapa de acesso da torre: cada ferramenta tem escopo de leitura e escrita declarado — e o agente que tenta ler além do escopo é bloqueado pelo harness. A conectividade vira governança, não burocracia.

O Modelo de Decisão de Aquisição de Ferramentas

O laboratório acumula ferramentas com facilidade — e cada uma cobra manutenção em contexto e configuração. O modelo abaixo é o gate de aquisição: antes de instalar uma nova ferramenta, a equipe responde a cinco perguntas e o modelo devolve o veredito:

```
def avaliar_ferramenta(nome, resolve_problema, substitutos, custo_contexto,
  curva_aprendizado):
    pontos = 0
    if resolve_problema:
```



```

    pontos += 3
    if not substitutos:
        pontos += 2
    if custo_contexto == 'baixo':
        pontos += 1
    if curva_aprendizado == 'curta':
        pontos += 1
    veredito = 'adquirir' if pontos >= 4 else 'avaliar' if pontos >= 2 else
'recusar'
    return {'ferramenta': nome, 'pontos': pontos, 'veredito': veredito}

print(avaliar_ferramenta('linter-ai', resolve_problema=True,
substitutos=['ruff'], custo_contexto='alto', curva_aprendizado='longa'))

```

O modelo é um antídoto para o acúmulo: ferramenta que resolve problema real, sem substituto, com custo de contexto baixo e curva curta entra direto; ferramenta redundante ou pesada fica de fora. A regra não é proibitiva — é seletiva. O laboratório deve ter poucas ferramentas excelentes, não muitas ferramentas medíocres, porque cada ferramenta instalada cobra aluguel em todo prompt futuro.

O Playbook de Diagnóstico de Motores

Quando o ecossistema falha, o Comandante diagnostica com método — não com tentativa e erro. O playbook abaixo é o roteiro de diagnóstico:

1. **Harness não executa?** Verifique o loop: ferramenta existe? Retorno chega ao modelo? Timeout?
2. **Skill não carrega?** Verifique o manifesto: o disparo bate com a fase? O arquivo existe?
3. **MCP não conecta?** Verifique o registro: escopo permite? Servidor está de pé? Autenticação?
4. **Merge conflita?** Verifique as worktrees: os agentes editaram o mesmo arquivo físico?
5. **Sessão estoura?** Verifique o orçamento: fase gastou além da alocação?

O playbook é o checklist do mecânico: cada sintoma mapeia uma causa provável e uma ação — o tempo de diagnóstico cai de horas para minutos.

O Modelo de Inventário de Skills com Custos

O ecossistema de skills cresce sem controle se ninguém mede. O modelo abaixo mantém o inventário de skills com custo de ativação e frequência de uso — o que expõe as skills que ninguém usa:

```

class InventarioDeSkills:
    def __init__(self):

```

```

    self.skills = {}

    def registrar(self, nome, custo_ativacao, categoria):
        self.skills[nome] = {'custo_ativacao': custo_ativacao, 'categoria':
categoria, 'usos': 0}

    def usar(self, nome):
        if nome in self.skills:
            self.skills[nome]['usos'] += 1

    def custo_total(self):
        return sum(s['custo_ativacao'] * s['usos'] for s in
self.skills.values())

    def obsoletas(self, limiar_usos=1):
        return {n: s for n, s in self.skills.items() if s['usos'] <=
limiar_usos}

inv = InventarioDeSkills()
inv.registrar('redigir-spec', 200, 'documentacao')
inv.registrar('auditor-legado', 1500, 'analise')
inv.usar('redigir-spec')
print(inv.obsoletas())
print(inv.custo_total())

```

A skill de auditoria legado com custo de ativação de 1500 tokens e zero usos é um desperdício vivo: cada prompt que a carrega sem usá-la é custo puro. O inventário responde duas perguntas: quanto o ecossistema custa por sessão (custo_total) e quais skills merecem revisão ou aposentadoria (obsoletas). O ecossistema enxuto não é sobre quantidade — é sobre relação custo-uso de cada skill registrada.

O Registro de Ativação de Skills

O manifesto declara o catálogo; o registro de ativação mede o uso. Cada carregamento de skill gera um evento — e o agregado desses eventos alimenta a governança do ecossistema:

```

{
  "ativacoes": [
    {
      "timestamp": "2026-08-02T10:12:00Z",
      "skill": "revisor-espec",
      "fase": "spec",
      "artefato": "spec_autenticacao.json",
      "resultado": "aprovado",
      "tokens_gastos": 4200
    }
  ]
}

```

```

    },
    {
      "timestamp": "2026-08-02T11:47:00Z",
      "skill": "revisor-espec",
      "fase": "spec",
      "artefato": "spec_pagamentos.json",
      "resultado": "reprovado",
      "tokens_gastos": 5100
    }
  ]
}

```

O registro responde perguntas que o manifesto não responde: qual skill é usada de verdade, qual falha com frequência, quanto custa cada ativação. É o contador de combustível da torre — a skill que custa caro e não entrega é candidata a revisão.

O Test-First Como Régua do Harness

O harness só está configurado de verdade quando o test-first é a régua: o agente abre o build pelo teste vermelho. O fluxo operacional é inegociável:

1. **Escreva o teste** que define o comportamento esperado (vermelho).
2. **Rode o teste** e registre a saída — a evidência do vermelho.
3. **Delegue a implementação** ao agente, com o teste como contrato.
4. **Rode de novo** — o verde é a prova de conclusão.
5. **Registre o par** (teste, saída) como evidência do merge.

O trecho abaixo é o esqueleto do contrato test-first que o harness valida antes de aceitar um build como concluído:

```

import subprocess
from pathlib import Path

def rodar_teste_com_contrato(teste: str, raiz: Path) -> dict:
    resultado = subprocess.run(
        ["python", "-m", "pytest", teste, "--quiet"],
        capture_output=True, text=True, cwd=str(raiz))
    return {
        "teste": teste,
        "exit_code": resultado.returncode,
        "saida": (resultado.stdout or resultado.stderr)[:200],
        "verde": resultado.returncode == 0,
    }

```

```
contrato = rodar_teste_com_contrato("testes/test_login.py", Path("."))
print(f"Teste {contrato['teste']}: {'VERDE' if contrato['verde'] else 'VERMELHO'}")
print(f"Saida: {contrato['saida']}")
```

O contrato é simples, mas muda o jogo: o agente não decide quando terminou — o teste decide.

O Ciclo de Vida de uma Skill

Skills não são eternas — nascem, são usadas, são criticadas e evoluem (você verá esse ciclo no Capítulo 8). O ciclo de vida técnico segue o mesmo padrão do SDLC que as skills governam:

Estágio	Ação	Evidência
Nascimento	Procedimento capturado de um incidente ou prática	Registro de origem
Uso	Skill carregada sob demanda em fases correspondentes	Log de ativação
Crítica	Taxa de sucesso medida por fase	Métrica de desempenho
Evolução	Skill revisada quando a taxa cai	Nova versão com changelog
Aposentadoria	Skill substituída por MCP ou procedimento superior	Registro de descontinuação

Cada skill no manifesto carrega essas métricas — a skill que falha três vezes seguidas vira insumo do debriefing, não dogma mantido.

O Modelo de Custo Total do Laboratório

O laboratório tem custo recorrente — e o comandante precisa do número. O modelo abaixo soma os custos de motor, ferramentas e contexto por sessão:

```
def custo_laboratorio(custo_motor_sessao, custo_ferramentas,
tokens_por_sessao):
    custo_tokens = tokens_por_sessao / 1000 * 0.002
```

```

total = custo_motor_sessao + custo_ferramentas + custo_tokens
return {'motor': custo_motor_sessao, 'ferramentas': custo_ferramentas,
        'tokens': round(custo_tokens, 3), 'total_sessao': round(total,
3)}}

print(custo_laboratorio(custo_motor_sessao=0.10, custo_ferramentas=0.02,
tokens_por_sessao=50000))

```

O número total por sessão alimenta duas decisões: quanto o laboratório custa por entrega e se a automação realmente compensa. Quando o custo por sessão supera o custo do trabalho manual que substitui, o laboratório virou passatempo caro — e o benchmark de motores da seção anterior é o que aponta onde cortar. Medir o custo é a disciplina que mantém o laboratório uma ferramenta, não uma despesa.

A Governança do Ecossistema

Quatro motores resolvem a execução, mas quem governa a combinação deles? A resposta é o contrato de delegação que você viu no Capítulo 2: cada motor tem um dono, uma régua e um ponto de auditoria. O harness é de quem opera a esteira; as skills são de quem mantém o procedimento; os MCPs são de quem administra o acesso a dados; as worktrees são de quem controla o merge. Sem essa governança, o ecossistema vira caos de ferramentas — o problema que o MCP justamente veio resolver ao padronizar o acesso.

O Radar do Ecossistema

A operação dos motores também precisa de observação. Métricas simples respondem se o ecossistema está saudável: taxa de sucesso por skill, latência por chamada MCP, conflitos por merge de worktree e tokens consumidos por sessão de harness. Essas métricas alimentam o debriefing do Capítulo 8 — o motor que falha três vezes seguidas vira skill revisada, não dogma mantido.

O Ecossistema em Números

Um laboratório de motores se avalia com números, não com opinião. Três indicadores mínimos: taxa de aprovação de código no CI (o harness está entregando sintaxe válida?), taxa de sucesso das skills (o procedimento empacotado está cumprindo o papel?) e conflitos por merge (as worktrees estão isolando de verdade?). Cada indicador alimenta o debriefing da Fase 8 — um motor que falha de forma consistente é candidato a skill revisada ou MCP substituído, nunca a costume mantido por inércia.

O Protocolo de Entrada de Nova Ferramenta

Nenhuma ferramenta entra no laboratório sem protocolo. O protocolo tem cinco etapas fixas: demonstrar que resolve problema real, comparar com os substitutos existentes, medir o custo de contexto por sessão, testar em um projeto piloto de baixo risco e documentar o caso de uso no inventário. As cinco etapas são obrigatórias e nessa ordem — a demonstração antes da instalação evita o entusiasmo prematuro, e o piloto de baixo risco evita que a estreia da ferramenta aconteça na entrega crítica. O protocolo não atrasa a adoção de boas ferramentas; ele só filtra as que não se sustentam.

Passos para Montar o Laboratório de Motores

1. **Escolha o harness** e configure o loop de execução (bash, edição, testes).
2. **Registre as ferramentas essenciais** como MCPs (filesystem, banco de estado).
3. **Empacote seus procedimentos em skills** — comece pelas duas de maior retorno: revisão de spec e verificação adversarial.
4. **Configure worktrees por agente** — um território por executante.
5. **Adote test-first como régua**: todo build começa pelo teste vermelho.

Aplica

Cena real, em segunda pessoa. Você lidera uma equipe de plataforma que decidiu delegar uma sprint inteira a agentes. A empolgação inicial vira caos na quarta-feira: dois agentes editam o mesmo arquivo de configuração e corrompem o build; o agente do front-end não encontra a API porque nenhum MCP conecta o harness ao serviço de documentação; e a revisão de código vira um gargalo porque o revisor humano precisa ler tudo sem ajuda.

O erro não foi usar agentes em paralelo. O erro foi montar os motores pela metade. Faltaram os quatro instrumentos do capítulo: worktrees (os agentes pisaram no mesmo território), MCPs (os agentes não tinham acesso aos dados), skills (cada agente improvisou seu procedimento de revisão) e a régua test-first (os agentes “terminaram” quando acharam que estava pronto, não quando os testes passaram).

O diagnóstico, ligado à teoria: motores faltantes viram conflito de execução. A correção prática:

1. **Pare a sprint e reconfigure**: uma worktree por agente, com branch própria e diretório próprio.
2. **Registre os MCPs de dados**: banco de estado, documentação, repositórios — o radar conectado.
3. **Empacote a skill de revisão** e a skill de build, e injete-as no contexto de cada agente.

4. **Exija teste vermelho antes de código:** o agente que não abre com teste não decola.

Armadilhas comuns: achar que harness é só o IDE (o harness é o loop, não a interface); registrar MCPs demais e transformar o contexto em colcha de retalhos (só o que a fase precisa); skills monolíticas que tentam cobrir tudo (skill especializada é skill que funciona); e worktrees sem merge controlado (isolamento sem integração é acúmulo de ilhas).

Conclusão

Você ligou os motores. Três marcos: primeiro, o harness como loop de execução com feedback — a anatomia que transforma LLM em agente; segundo, as skills e MCPs como especialização e conectividade — procedimentos empacotados e dados sob demanda via protocolo padrão; terceiro, as worktrees como célula de contenção do build paralelo e o test-first como régua mensurável de conclusão.

Como desafio, configure uma worktree isolada para o seu próximo experimento agêntico e escreva o teste vermelho da feature antes de qualquer prompt ao agente.

No próximo capítulo, você aciona o radar: verificação adversarial e evidência — a camada que separa o SDLC AI-first de um caos com boa intenção.

Por que este capítulo importa

Se você chegou até aqui, já percebeu que os motores: harness, skills, mcps e worktrees. Este capítulo — *Capítulo 5: Os Motores: Harness, Skills, MCPs e Worktrees* — é um convite para parar de usar IA como ferramenta de autocomplete e começar a tratá-la como parte estrutural do seu processo de desenvolvimento. A diferença entre as duas posturas é exatamente o que separa quem apenas acelera o que já fazia de quem repensa o que é possível.

Vamos explorar isso com exemplos práticos, código real e um passo a passo que você pode aplicar ainda hoje, sem esperar por infraestrutura nova ou aprovação de comitê. A ideia é simples: cada seção termina com algo que você pode executar em menos de uma hora.

Conceitos-chave deste capítulo

- **O contrato antes da execução:** antes de deixar qualquer agente trabalhar, defina o que significa “feito” em termos verificáveis.

- **Evidência antes de afirmação:** o que não pode ser verificado não pode ser delegado com segurança.
- **Aprendizado contínuo:** cada entrega, boa ou ruim, é matéria-prima para o próximo ciclo.

Esses três conceitos aparecem, de formas diferentes, em todas as seções a seguir. Mantê-los em mente enquanto você lê vai transformar exemplos isolados em um padrão que você reconhece na sua própria rotina.

Checklist para aplicar hoje

1. Escolha uma tarefa pequena e bem definida do seu backlog.
2. Escreva o critério de aceite em uma frase verificável.
3. Delegue a execução a um agente, mantendo a verificação em suas mãos.
4. Registre o que funcionou e o que não funcionou.
5. Transforme o aprendizado em um procedimento reutilizável.

Se você fizer apenas o primeiro item, já estará à frente da maioria das equipes — que continua discutindo IA em reuniões sem nunca definir o que quer que ela faça.

Perguntas que você deve se fazer

1. Qual fase do meu processo consome mais tempo hoje — e por quê?
2. O que eu delegaria a um agente amanhã se tivesse certeza de que o resultado seria verificado?
3. Qual informação eu poderia registrar hoje que tornaria a próxima iteração mais barata?
4. Quem no meu time revisa o trabalho de quem — e com qual critério?
5. O que eu faria se o custo de cada tentativa caísse para quase zero?

Essas perguntas não têm resposta certa, mas têm uma propriedade em comum: elas forçam você a sair da conversa abstrata sobre IA e entrar no terreno do seu processo real. E é exatamente nesse terreno que o SDLC AI-first produz resultado.

Glossário rápido

- **Agente:** programa que usa um modelo de linguagem para planejar e executar tarefas com acesso a ferramentas.
- **Harness:** a camada que conecta o agente ao ambiente — arquivos, comandos, testes e regras.
- **Spec executável:** especificação cujos critérios podem ser verificados por máquina.

- **Verificação adversarial:** camada que refuta o trabalho produzido, em vez de apenas confirmá-lo.
- **Contexto:** a janela de informação que o modelo enxerga a cada passo — o recurso mais caro do ciclo.

Dominar esses cinco termos é suficiente para acompanhar qualquer discussão séria sobre desenvolvimento orientado a agentes.

O erro mais comum nesta fase

A maioria das equipes comete o mesmo erro: adota a ferramenta e mantém o processo. O agente entra no fluxo como um autocomplete sofisticado, e todo o potencial de transformação se perde em pequenas conveniências. O antídoto é simples e desconfortável: mude o processo primeiro, depois traga a ferramenta. Defina o contrato, o critério de aceite e a verificação antes de permitir que o agente produza em escala. É contra intuitivo, mas é o que separa as equipes que capturam valor das que apenas geram volume.

Um exemplo concreto para fixar

Imagine uma pequena feature de faturamento. No fluxo tradicional, um desenvolvedor recebe a tarefa, interpreta a intenção, escreve o código e um revisor confia na leitura. No fluxo orientado a agentes, a mesma feature começa com uma frase verificável: “o valor total deve considerar o desconto aplicado antes dos impostos”. O agente implementa, os testes verificam a regra, e o humano revisa a evidência — não o código linha a linha, mas o comportamento observado. Perceba o deslocamento: o humano deixa de ler tudo para auditar o essencial, e o agente deixa de adivinhar para executar contra um critério. É essa troca que o restante do livro explora em profundidade.

A rotina de quem já opera assim

Uma semana de trabalho em uma equipe que já adotou o ciclo orientado a agentes não parece uma revolução — parece um fluxo calmo e bem definido. Na segunda-feira, a especificação da semana é revisada em uma reunião curta: cada critério de aceite é lido em voz alta e qualquer ambiguidade é resolvida antes de tocar em código. Na terça, os agentes executam as tarefas em isolamento, enquanto os humanos revisam a arquitetura e os contratos. Na quarta, a verificação

roda: testes, revisão adversarial e a decisão de merge apoiada em evidência. Na quinta, o que passou vai para produção em canário, com observabilidade ligada. Na sexta, o debriefing transforma os incidentes da semana em lições e skills. Nenhum dia é heroico; todos os dias são previsíveis. E é exatamente essa previsibilidade — não a velocidade máxima — que define a alta performance.

O que não fazer: os anti-padrões mais comuns

Se você quer destruir o valor do desenvolvimento orientado a agentes, aqui estão as receitas mais eficientes. Primeiro, o prompt-and-pray: gere o código, olhe por cima, e peça desculpas quando quebrar. Funciona em demos, falha em produção. Segundo, a spec decorativa: escreva documentos longos que ninguém verifica e que o agente não consegue executar — o pior dos dois mundos. Terceiro, a auto-verificação: deixe que quem escreveu valide o próprio trabalho, sem revisor independente; é a forma mais rápida de transformar confiança em acidente. Quarto, a delegação sem observabilidade: conceda autonomia sem instrumentar o comportamento. Todos esses padrões têm uma origem comum — a pressa em capturar o ganho sem construir o controle. E todos têm o mesmo antídoto: contrato antes de execução, evidência antes de afirmação, e revisão independente em toda entrega.

Como medir o progresso na prática

Uma dúvida legítima é: como saber se a adoção está dando certo? Métricas tradicionais de velocidade podem enganar — um time pode entregar mais rápido e acumular dívida técnica invisível. O indicador mais confiável no ciclo orientado a agentes é a estabilidade: quantos incidentes em produção, quanto tempo de retrabalho, quantas correções de emergência. Um segundo indicador é o custo de contexto: quantos tokens cada fase consome, e onde o desperdício se concentra. Um terceiro é a taxa de aceite na primeira verificação: se os agentes precisam de muitas rodadas de refutação, o contrato está fraco — o problema não é o agente, é a spec. Com esses três números na mesa, a conversa de progresso deixa de ser anedótica e vira análise de processo.

O papel do líder neste capítulo

Nada do que este capítulo descreve acontece por acaso — alguém precisa criar as condições para que o processo exista. Esse alguém é o líder técnico, o líder de equipe ou o arquiteto que

decidiu tratar o ciclo orientado a agentes como uma mudança de processo, não como a instalação de uma ferramenta. O trabalho do líder aqui tem quatro frentes. A primeira é a modelagem do contrato: garantir que cada fase tem entrada, saída e critério definidos. A segunda é a calibragem da confiança: decidir o que pode ser delegado e o que exige decisão humana, e documentar essa decisão. A terceira é a defesa do tempo de verificação: em uma cultura que celebra velocidade, o líder precisa defender o orçamento de revisão como quem defende o seguro do prédio. A quarta é o exemplo: o líder que pede evidência antes de afirmar, em toda reunião, ensina mais do que qualquer documento de processo.

Perguntas frequentes honestas

P: Isso não vai tirar o emprego dos desenvolvedores? R: A história do ciclo de vida nunca foi sobre menos trabalho humano, mas sobre trabalho mais valioso. O que muda é a natureza da tarefa: escrever código repetitivo deixa de ser o centro, e a especificação, a verificação e o desenho de contratos ocupam o lugar. P: Precisamos de um time de especialistas em IA para começar? R: Não. Precisa-se de disciplina de processo e de vontade de medir. As ferramentas evoluem rápido; o processo é o que permanece. P: E se o agente produzir código que ninguém entende? R: Essa é a pergunta certa — e a resposta é a verificação: se o código passa nos testes, na revisão adversarial e na observabilidade em produção, o fato de ter sido escrito por um agente é irrelevante. O critério não é a origem, é a evidência. P: Quanto tempo leva para ver resultado? R: Na primeira semana você já vê o efeito de escrever critérios de aceite verificáveis, independentemente de agentes. Os ganhos estruturais aparecem em um a dois ciclos.

Um convite para a prática deliberada

Conhecimento sem prática é entretenimento disfarçado de aprendizado. Este capítulo termina com um convite para a prática deliberada: escolha um artefato real do seu trabalho — uma spec, um teste, um release — e aplique deliberadamente um dos conceitos aqui descritos. Anote o antes e o depois. Repita por quatro semanas. No fim do mês, compare: o processo está mais previsível? O custo de contexto caiu? A estabilidade melhorou? Esse experimento pessoal, pequeno e mensurável, vale mais do que qualquer curso. É assim que o ciclo orientado a agentes deixa de ser um conceito que você explica para outras pessoas e se torna uma capacidade que você demonstra.

Síntese para levar com você

Se você guardar apenas uma ideia deste capítulo, que seja esta: no ciclo orientado a agentes, o contrato precede a execução, a evidência precede a afirmação e a revisão independente precede a entrega. Tudo o mais — as ferramentas, os modelos, os fluxos — muda rápido e pode ser aprendido conforme a necessidade. O que não muda é a disciplina: sem ela, a IA é um gerador de volume; com ela, é um multiplicador de capacidade. O resto do livro é a expansão dessa disciplina em cada fase do ciclo de vida.

De onde veio a necessidade desta mudança

Vale a pena entender por que este capítulo existe — e por que ele não foi escrito dez anos atrás. A resposta está na economia do desenvolvimento de software. Durante décadas, o custo dominante de produzir software foi o trabalho humano: escrever, revisar, corrigir. Todo o ciclo de vida clássico foi desenhado em torno dessa escassez — processos, papéis e artefatos existem para coordenar pessoas e evitar retrabalho caro. O que mudou nos últimos anos foi a emergência de modelos capazes de gerar, revisar e executar código com custo marginal próximo de zero. De repente, a escassez dominante não é mais a mão de obra: é a capacidade de especificar, orquestrar e verificar. Esse deslocamento — de horas-homem para tokens e contexto — é a raiz de tudo o que este capítulo descreve. Quem entende essa mudança de economia entende por que o processo precisa mudar junto com a ferramenta.

Conversando com quem resiste

Em toda equipe há quem resista à mudança — e a resistência quase nunca é preguiça, é uma pergunta legítima sem resposta. As objeções mais comuns são três. “Já tentamos automação e quebrou”: a resposta é que a automação anterior quebrou porque o processo não tinha contrato nem verificação; é exatamente isso que o novo ciclo constrói antes de automatizar. “IA gera código que ninguém entende”: a resposta é que o critério de entendimento mudou — o que importa não é a origem do código, mas se ele passa na verificação; e a revisão de contrato e arquitetura continua humana. “Isso é modismo”: a resposta mais honesta é que pode ser, mas o processo que este capítulo descreve — especificar, delegar, verificar, aprender — melhora o ciclo com ou sem IA. A disciplina é o investimento à prova de modismo.

O dia a dia no detalhe

Para tornar concreto o que este capítulo descreve, vale percorrer o dia a dia de uma tarefa típica, passo a passo. A manhã começa com a revisão da intenção: o produto explica o que quer, o time traduz em requisitos com critérios verificáveis e ninguém toca em código antes de a spec estar aprovada. Na sequência, o trabalho é despachado: cada tarefa vai para um contexto isolado, com seu contrato anexado. A execução produz artefatos — código, testes, diagramas — e cada artefato carrega a evidência de como foi produzido. A tarde é de verificação: testes automáticos, revisão adversarial e a leitura humana do que é crítico. O que passa, segue; o que não passa, volta com o parecer anexado — sem discussão de opinião, porque o critério já estava escrito. O fim do dia é de registro: o que foi aprendido, o que custou em contexto, o que deve mudar no processo. Esse fluxo parece simples, mas cada passo exige disciplina — e é exatamente a simplicidade do ritmo que o torna sustentável.

O custo invisível que decide tudo

Há um recurso que atravessa todos os exemplos deste capítulo e que raramente aparece nas discussões: o contexto. Cada interação com um modelo de linguagem consome uma janela de informação — e essa janela é limitada e cara. Uma spec mal escrita gasta contexto em ciclos de correção. Um log inteiro no contexto gasta contexto que poderia servir à verificação. Uma busca redundante gasta contexto sem produzir informação. Quem ignora esse custo descobre, cedo ou tarde, que a automação ficou mais cara que o trabalho manual que pretendia substituir. Por isso a disciplina de contexto não é um detalhe de economia — é uma decisão de arquitetura do ciclo. Medir o consumo por fase, comprimir o que é ruído e injetar apenas o necessário são práticas que determinam se o SDLC AI-first se sustenta em escala. Este capítulo toca nesse tema; os capítulos finais do livro o desdobram em técnica.

Uma visão de longo prazo

A adoção do ciclo orientado a agentes não é um projeto com data de fim — é uma trajetória que se desenrola ao longo de anos, e vale a pena olhar para a frente. No primeiro trimestre, o foco é o contrato: as equipes aprendem a especificar e a verificar, e os ganhos vêm da clareza, não da automação. No segundo trimestre, a delegação supervisionada entra em produção em áreas de baixo risco, e as métricas começam a mostrar onde o ciclo ganha e onde perde. No segundo ano, o ciclo adversarial se consolida: verificação automática, observabilidade do comportamento agêntico e aprendizado organizacional rodando como rotina. No terceiro ano, a organização

opera em um nível de maturidade em que a IA é parte estrutural do processo, e a pergunta deixa de ser “como adotar” e passa a ser “como evoluir”. Quem inicia com o processo antes da ferramenta chega a esse destino com estabilidade; quem inverte a ordem, chega com dívida. A escolha é sua.

Próximos Passos

Você acabou de percorrer um dos capítulos centrais do SDLC AI-first. Se este conteúdo fez sentido para você, o próximo passo natural é continuar a jornada pelo livro completo, que aprofunda cada fase do ciclo: especificação executável, design de domínio, harness e agentes, verificação adversarial, entrega segura, aprendizado contínuo, economia de tokens e governança de maturidade.

Enquanto isso, aqui vão três ações concretas:

1. **Pratique em um projeto pequeno.** Nada substitui experimentar com as próprias mãos — escolha um repositório pessoal e aplique um dos conceitos deste capítulo.
2. **Formalize seu contrato.** Escreva o critério de aceite de uma tarefa real antes de delegá-la. É um exercício de cinco minutos que muda completamente a qualidade do resultado.
3. **Mensure seu processo.** Registre quanto tempo e quanto contexto cada fase consome. O que não é medido não pode ser melhorado.

O céu do software agora tem torres de controle. O próximo voo é seu.

Boa leitura e bons voos.

— Heverton Eduardo Peres.

